

On Premises IIoT Architecture for Capturing, Storing and Visualizing Production Data

Fabian Hensler

Group Manufacturing Support
Würth Elektronik eiSos
Waldenburg, Germany
fabian.hensler@we-online.de

Alexander Jesser

Institute ICPS
Hochschule Heilbronn
Künzelsau, Germany
alexander.jesser@hs-heilbronn.de

Heinz Frank

Hochschule Heilbronn
Künzelsau, Germany
heinz.frank@hs-heilbronn.de

Nico Strebe

Group Manufacturing Support
Würth Elektronik eiSos
Waldenburg, Germany
nico.strebe@we-online.de

Marius Kappler

Product Management
Würth Elektronik eiSos
Waldenburg, Germany
marius.kappler@we-online.de

Steffen Mütsch

Product Management
Würth Elektronik eiSos
Waldenburg, Germany
steffen.muetsch@we-online.de

Abstract—In this paper an automated system for production data acquisition, storage and analysis is proposed. It covers the structure and a test of the entire data path consisting of machines, data broker, database and visualization software. Here, unlike other solutions, the actual data source in the production process is variable and not limited to a specific protocol or connector. To achieve this, a locally available gateway running the software Node-RED is used, to receive the data from the different machines and forward it to a central server using MQTT, for example. On this server, the data is stored in the time series database InfluxDB and is consequently available for analysis via Grafana. A particular focus lies on the independent software components, which can be easily exchanged and combined in other ways. Thus, choosing this software means that you won't be affected by the vendor lock-in that is common in the market. Security wise each software component can be hosted on premises, which is a deliberate move away from the cloud solutions that are common in many places. In addition to the structure of the described system, this paper also deals with the performance in terms of writing data to or retrieving data from the database compared to other solutions. Moreover the latency of different approaches in the system is considered.

Index Terms—Data mining, MQTT, Node-RED, InfluxDB, Grafana

I. INTRODUCTION

Industry is in a state of flux. Since the beginning of the 19th century, the way of producing has been constantly evolving. Over several industrial revolutions, optimized mechanical processes, the integration of electrics and digitalization found their way into production halls. Today, we are in the midst of the fourth industrial revolution, through which the networking of production facilities is in the foreground. Among other things, the associated improvements mean that the individualization of products can be increased through highly flexible processes, even in the context of mass production, as described in [1]. This circumstance leads to a diverse range of networking solutions from different suppliers on the market. In particular, combining all production-relevant machines, sensors, inventories, etc. in a single Manufacturing Execution System (MES)

has become a desire of many manufacturing companies. When it comes to developing an MES in line with the fourth industrial revolution, aspects such as decentralization, mobile connectivity and cloud computing must be combined [2].

Systems, as shown in [3] by the Bosch Rexroth company, are increasingly establishing themselves on the market. In many cases, the supplier tries to retain customers by offering a big range of products in order to deliver a single system solution. Although examples like from Bosch Rexroth show how feature-rich and sophisticated such proprietary solutions can be, they do have some drawbacks. These include dependencies and security concerns, especially when it comes to data protection.

Due to the extensive digitalization of many processes in both the private and business environment, this topic has become significantly more important in recent decades. Particularly in Europe, but also in the U.S., the focus is no longer on voluntary caution, but on corresponding laws to protect sensitive data, as mentioned in [4].

For this reason, the present paper presents an IIoT system in which the collected data moves exclusively within the own network. No cloud services from external providers are required. In addition, the communication between the individual components is encrypted via TLS. Many major machine protocols are supported to ensure maximum compatibility. It is even possible to combine this system with already existing software or successfully established processes. To give an example, it is possible to allow a company's supply chain management access to the collected data. The employees can improve the availability planning based on the provided information. At the same time, the software modules can be flexibly combined and adapted to the circumstances. The software used is under open source license and thus enables an easy start without high financial expenditure. However, when considering a long-term and more professional implementation, it is recommended to upgrade to the software's enterprise version.

The Paper is organized as follows: After this introduction with Section II some background and related work is stated. In the next Section III the structure and all possible variants of the proposed system with their links are explained in detail. Section IV then deals with the results obtained from this design. The paper is rounded off by the conclusion in Section V.

II. BACKGROUND AND RELATED WORK

As described prior, the system consists of three main components:

- Node-RED
- InfluxDB
- Grafana

Even if the system is basically made up of these three components, there is the possibility of expansion. The basic function can therefore be expanded by data transmission via MQTT or by integrating a streaming platform. Both are particularly relevant for larger scaled implementations, as described in the further course of this paper.

The characteristics of these main software components and their use in comparable applications will now be carried out below.

A. Node-RED

According to [5], Node-RED is a flow-based programming tool which belongs to the OpenJS Foundation since 2019. Previously it was developed by IBM for six years. The entire open-source application is based on Node.js and can be accessed through a web interface with which the developer is able to combine and connect *nodes* representing a specific function. Hence, with this Low-Code/No-Code kind of programming it is easy to implement IoT procedures quite fast, as has been shown in [6] and [7].

Also remarkable is the ability of Node-RED to write user defined JavaScript-Code in so called *function nodes* and the possibility to install new nodes out of a library. In this way the developer is able to adapt Node-RED to several types of IoT protocols, as it was described in [8] with Modbus, for example.

B. InfluxDB

InfluxDB is an open-source time series database (TSDB) which provides the user a SQL-like query language. TSDB means that the database is optimized to handle time series data like metrics or measurements. In the wake of the IoT movement, this type of database has become increasingly popular as more devices produce time series data. On the other hand, there are relational databases that are still used, but for different purposes. Many big tech companies are switching over to TSDB for monitoring purposes [9].

As shown in [10], InfluxDB is one of the most popular TSDBs due to its versatile application possibilities. In addition to the actual database, the supplier InfluxData also offers other software packages for capturing, displaying and processing data.

C. Grafana

The open-source visualization and alerting software Grafana is specialized in queueing data from TSDBs. Compatible databases include Prometheus, InfluxDB, but also SQL databases like MySQL. Therefore, Grafana is able to unify many types of databases in order to retrieve and visualize data. This can be done through a web interface where the user can set up multiple dashboards with different panels like time series, bar charts, etc. All functionalities, as well as the alerting tool, are described more detailed in [11].

The mentioned features make Grafana a suitable software in the IoT area. It has already been used many times in the past to visualize recorded metrics. One example for the industrial sector is [12], where data was obtained from PLCs, stored and visualized.

III. SYSTEM DESIGN

The system structure is presented in chronological order from data source to data sink. Fig. 1 shows the schematic composition of the introduced software.

As mentioned before, the actual data source is variable. It can be any type of device in a production plant that provides suitable data. Machines whose integrated controllers already support appropriate protocols such as OPC UA or Modbus come into question, for example. But also various PLCs or network-capable sensors, which are used e.g. for the external acquisition of environmental parameters, are possible, just to name a few.

In the next step all the data is collected by the software Node-RED. The most simple implementation is under a Linux based operating system like Debian or Ubuntu. It can be hosted on a locally available server at the production plant or even on a simple industrial PC. Virtualization is also possible to reduce management and security overhead. But it is recommended to run it close to the manufacturing equipment in the LAN to avoid connection issues due to high latency or disconnections. Another reason for this is the inability of many production devices to provide accurate timestamps for the data they generate. Some devices are not capable to deliver timestamps at all and some are just very imprecise because they are not time synchronized. In contrast to that, the system running Node-RED can be time synchronized through NTP or SNTP and thus generate a timestamp each time new data is retrieved. Noteworthy is also the ability of Node-RED to secure it with TLS and an user authentication. So in order to encrypt ingoing and outgoing communication it is possible to implement certificates. This is relevant for the secure access via HTTPS to the web interface of the application as well. As for the user authentication specific permissions like *admin* or *read only* for different users can be set. In addition to a user name, a password is assigned. This password is saved in the configuration file as a bcrypt hash. Therefore, it is secured even if an attacker gets access to the file.

The further data path depends on the use case, since it is possible to send data into two different directions. The first opportunity would be to install a local InfluxDB database

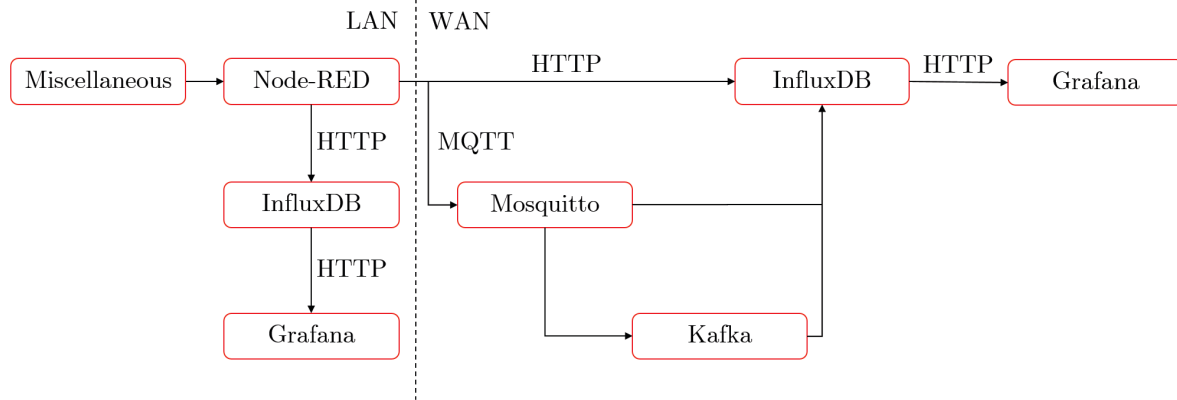


Fig. 1. Schematic software structure.

and also a local Grafana instance possibly even on the same server as Node-RED. Then forward the data through Node-RED via HTTP to the database, for which Node-RED provides corresponding nodes. After that the user can retrieve the data in Grafana also via HTTP. Hence, the data is available at the production plant and can be queued without a high latency. In deciding whether the data should remain in the local network or be sent on to a higher-level hierarchy, various scenarios are now conceivable. For example, all data accumulating in this LAN can be collected and stored locally while also being forwarded. Alternatively, only certain, particularly critical data can be stored locally, or a specific time can be defined for which the data is stored locally. Data with an older timestamp is thus automatically deleted over time.

Now looking at the forwarding of data to the WAN in more detail, it can be done in the global network of the company or the institution. By doing so it is recommended to use a VPN based connection and TLS in order to secure the transmission. When it now comes to send data globally, especially with an increasing amount of payload, the used protocol is getting more relevant. In this context the IoT protocol MQTT is reasonable. According to [13], HTTP uses 3 times more bandwidth compared to MQTT without any payload. With payload HTTP is still 2.5 times higher. Thus it is justified to call MQTT more suitable for WAN applications, due to the higher latencies occurring there with lower bandwidths at the same time. Although it is theoretically possible to use HTTP like shown in Fig. 1, it is not recommended.

Another important ability of MQTT is to organize different messages in specific topics. Therefore the user can set up a hierarchical structure to label the data with important metadata like for example:

- country
- location
- name of production
- name of device

These information is separated with the slash character. So the structure could be *country/location/name of production/name*

of device. Since MQTT is working with a publish/subscribe architecture each publisher can publish data into a topic and each subscriber can subscribe to a topic in order to get the published data. For the MQTT communication a MQTT broker is mandatory. In this work the broker Mosquitto has been used, which is a small open-source software. As described before the entire system structure is very flexible. Hence, it is possible to replace the Mosquitto with a more professional broker offering important tools like clustering for high availability.

In order to forward the data further it need to be subscribed. To pass the data directly to InfluxDB, the Telegraf software can be used, which is also part of the InfluxData software stack. Alternatively a data streaming platform like the open-source software Kafka can be used combined with Telegraf, depending on the use case. Assumed Telegraf runs on the same server or at least the same LAN as the database does, it can subscribe the data via MQTT and insert it into the database via HTTP without any problems in the long run. Since this is happening on the same server the MQTT protocol is not that relevant any more. If the data path consists of Apache Kafka, it is also Telegraf that can consume the data from Kafka and put it into InfluxDB. So the destination is equal on both paths and both are using the Telegraf software that is not shown in Fig. 1 for the sake of clarity and because of the fact that it belongs to the Influx software stack.

The decision between the proposed opportunities depends on the desired reliability. Since Kafka delivers a loosely coupled data transmission it ensures higher safety, near real-time and a much more scalable streaming, as can be seen in [14].

However, it all ends up in the TSDB InfluxDB. The reason why it is so important to use a TSDB in these timestamped IoT scenarios will be shown in the next section. Worth mentioning is the ability of InfluxDB to set multiple users with specific permissions. It can be used to restrict the access to the database so that the user only sees permitted content. Additionally the rights to read or write can be defined user independently. Last but not least InfluxDB offers the possibility to encrypt transmissions via TLS which is a great security advantage.

The data format in the database is as follows: Since InfluxDB is a TSDB, a field with a *timestamp* is mandatory. If data written to InfluxDB does not contain a *timestamp*, one is automatically generated at the time the row is created. The time is stored in the UNIX format which corresponds to the number of seconds since the first January 1970. However, smaller time specifications such as milliseconds and microseconds etc. are also possible. The first November 2022 at 1pm would give the number 1667304000 according to Central European Time (CET). Speaking of time zones, it is important to make a uniform reference to a specific time zone in the database as soon as data from different time zones are collected. If this is not done, it will lead to a lot of confusion in data analysis. A suitable time zone for an international overview is the Coordinated Universal Time (UTC). This is therefore used for all entries in the presented system.

In the same row, there must also be a *field key* and a *field value*. The *field key* describes the value of the *field value*. To give an example, a suitable *field key* would be *temperature* and a field value 22, which stands for 22 degrees Celsius. Here again, it is advantageous to standardize data sets, i.e. in this case to store the data in the base unit. So instead of writing a data point with, for example, the unit milliwatt into the database, it is converted into the unit watt. The value changes accordingly. The *field value* can be of different data types, such as binary, string, float, etc.

Optional are tags, which in turn consist of *tag key* and *tag value*. These are used to tag the data with metadata like the location, the sensor name or other declarations. The principle with key and value is similar to the fields. Again, to give an example, a valid *tag key* would be *country* and a associated *tag value* would be *Germany*.

Multiple rows are then concluded in a *measurement*. A *measurement* describes for example the sensor which provides the data. This is comparable to a table in an SQL database.

In order to provide a better understanding of the described database structure, it is once again shown graphically in Fig. 2. In the proposed system InfluxDB 1.8.10 was used although newer releases were available. This is because the functionalities of the older version are sufficient in this use case and the SQL-like query language of the command line tool is quite similar to MySQL so it is easy to adapt.

At the very end of the data path there is the visualization software Grafana. It connects directly to the InfluxDB via HTTP and retrieves the data. Since the database and Grafana are in the same LAN the use of HTTP is reasonable. It is important to know that access to the database through Grafana is read-only. Hence, a Grafana user is not able to change data in the database. However, he can download data through the web browser in a .csv format for offline analysis. Similar to the InfluxDB Grafana is also capable of setting users with their own credentials and restrictions. Therefore you can secure the data. Another security feature here is also TLS encryption.

To summarize this section, it is important to note that any software used supports TLS. Thus it is possible to create a encrypted network communication from the data source

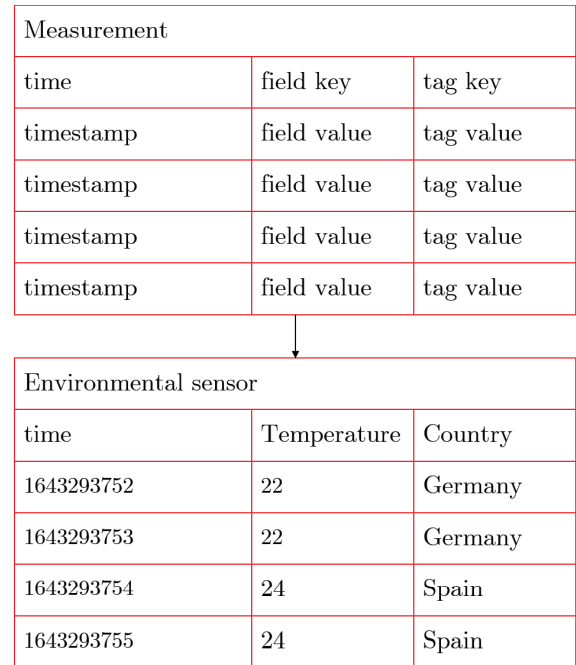


Fig. 2. InfluxDB data structure.

in the production to the data destination on the Grafana dashboard. However, one possible exception could be the sensor or the machinery at the very beginning of the data path. As described previously this part of the system is variable. Consequently, it is not possible to make a general statement in this regard. Some protocols like OPC UA are able to establish an encrypted connection to Node-RED but it also depends on the connected device itself. Some others especially older protocols do not have these functionalities. In these cases it is highly recommended to hide the communication in a separated VLAN. Even though it is particularly important in the cases mentioned, this procedure is equally suggested for all other connections near the machine.

IV. RESULT

The LAN version of the system is running without any problems for more than five months now. Also the WAN alternative over MQTT but without Kafka was build up and is running constantly for over three months now. The remaining data paths were set up for smaller tests but were not used in the long run.

Fig. 3 shows an exemplary section of the Grafana dashboard. This is one possible way of visualizing data retrieved out of a injection molding machine. This machine is connected via OPC UA to Node-RED. Node-RED itself is running on a industrial PC which is located at the production plant. Then the data is transmitted through MQTT to the broker running on the headquarters' servers so at a different location. Finally all the data is transferred to the database as proposed earlier. As can be seen in the figure, the parameter *mass cushion actual* is

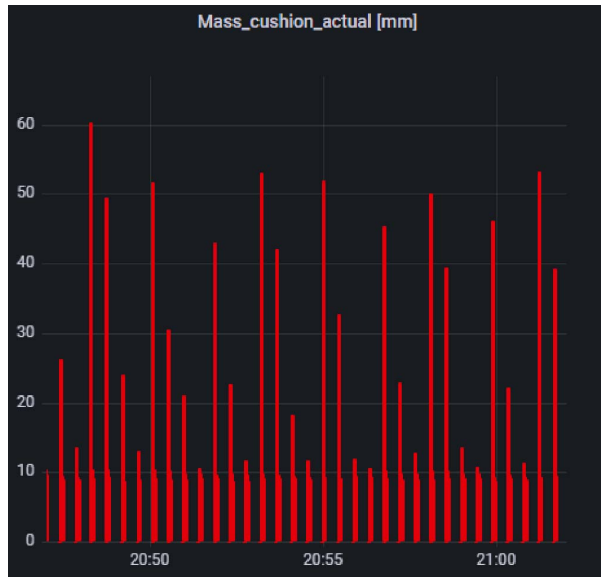


Fig. 3. Captured data on the Grafana dashboard.

selected. *Actual* means in this case that it is the current value measured at exactly the shown time. In addition, there is also a target value which is not in this diagram. At this point it should be added that this is an example and all components such as scaling, colors, type of diagram, etc. are modifiable. In fact, this injection molding machine offers the possibility of reading out far more parameters. Numerous internal sensors are managed by the integrated machine control system and are thus available for external connection. Which parameters are worth recording depends on the respective use case. If, for example, certain error patterns occur which indicate that the temperature is too low, this should be recorded. It is always advisable to seek the advice of experts for precisely this machine or process. They know best which parameters are important.

Approximately every 26 seconds a new part gets molded. Therefore these very close data points are related. They are all from one interval in which the mass cushion builds up. After a short break a new cycle begins and so on.

As already mentioned the presented parameter is one out of many available by the machine but particularly important for the quality. With this data basis, further work can now be done, for example, to better recognize error patterns or to optimize maintenance cycles. It is also conceivable to work with artificial intelligence methods on the data collected here. These examples only give an idea of the possibilities opened up by this.

Regarding security and TLS encryption Fig. 4 shows a comparison between an unencrypted and an encrypted MQTT transmission. It can be seen that no encryption has yet been performed in the upper half of the image. As a result the transmitted text to the right of the HEX code can be deciphered

without any problems. In contrast, below there is an intercepted transmission that has been encrypted. Consequently, the transmitted text to the right cannot be decoded without a key anymore. This example shows how important state-of-the-art encryption of data traffic is. Even if the communication should take place in the local network, this is important in order not to offer attackers who have made it into one's own network any attack surface.

Furthermore, the selection of the database of an IIoT system is very important, as it can have a very large impact on the performance of the overall system. To show how big the differences can be and why the InfluxDB database was chosen in this work, a synthetic benchmark was performed. In order to be able to test only the software itself, two identical virtual machines were set up. Both have been allocated the same hardware resources including network capacities. In this case it corresponds to an Intel i3-6100 with two cores and 4 GB RAM and a Gigabit Ethernet connection assigned to the VMs. Then InfluxDB was installed on one VM and MySQL on the other. MySQL was chosen as a classic representative of a relational database. This means that in this benchmark MySQL is a representative of relational databases and InfluxDB is a representative of time series databases. In order to write and read data to or from a database and record the time needed a Python script was developed in both cases.

The same data was written in both cases, consisting of a timestamp, a temperature indication and information about the location in one row. While the specification of the location remained the same, the temperature value changed according to a random function. A new timestamp was created each time according to the real time. To avoid duplicate timestamps in the program flow, a delay of 3 ms was provided after each generated row.

Fig. 5 now shows the deviation when it comes to writing data. The x axis shows the amount of rows written and the y axis shows the time required. Each test was performed 10 times and after that the arithmetic mean was used to plot one data point in the diagram. In blue color the data of the MySQL database can be seen. The InfluxDB data are presented in four separate curves. This is because InfluxDB supports batch writing of data. The number in the legend indicates the batch size of the performed test. In contrast, MySQL does not provide such a feature. It is only possible to define a packet size which was the default value of 67108864 during the benchmark. Increasing this value did not lead to any improvement in performance. However, the function *mycursor.executemany* was used to write a whole tuple of data in one step with which a clear increase in performance was noticeable for the MySQL database.

Looking at the graphs, it can be seen that all are starting quite close together when writing one data point. It can be stated that this is the minimum of the test system. It is also observable that the increase of data leads to an approximate proportional increase of needed time after the amount of data gets high enough to not let the script write all data at once into the database.

but theoretically possible, it is excluded in this test. Hence, a path from Node-RED directly to InfluxDB in a local network is one investigated option. The second opportunity would be a WAN connection through MQTT with Mosquitto as a MQTT broker and InfluxDB as the following component. Last but not least there is a similar path to the second but with Kafka between the MQTT broker and InfluxDB.

Now the latency of the three possibilities shall be considered. When it comes to a highly scaled and international IoT structure this parameter is of great importance. Especially when automated decisions must be made in real time the system is relying on a fast access to the collected data.

Therefore the three proposed data paths were set up and the same data as used in the benchmark before consisting of a timestamp, a temperature and a location was sent from Node-RED. This data was then sent with an interval of three seconds ten times and the latency was recorded. After that the arithmetic mean of all ten attempts was calculated. In order to retrieve the latency two identical data packages were sent at the same time. One contains a timestamp generated by the Raspberry Pi's system clock and one does not. InfluxDB as a time series database at the end stores the data package with the timestamp just like it is. But for the package without a timestamp it will automatically generate one based on the VM's system clock at the exact time it receives the data. After this procedure two rows of data are in the database but with slightly different timestamps. The latency is then just the subtraction of the InfluxDB timestamp minus the Node-RED's timestamp.

Of course this presupposes that the system clock of the Raspberry Pi and the VM must be well synchronized. For that purpose NTP was used. On both systems the same NTP server has been configured and the synchronization has been enabled before tests were made. As stated in [16], the typical accuracy of NTP is about 5 ms to 100 ms depending on the network. So it is definitely something to keep in mind when discussing the results. But since the relative relationship between the results is more important than the absolute numerical values, this is tolerated.

In order to create two hardware independent systems a Raspberry Pi 4 was used to simulate a gateway running Node-RED in the version 3.0.2. The InfluxDB is still used in the version 1.8.10 on a VM running on an Intel i3-6100 with two cores and 4 GB RAM assigned. The Gigabit Ethernet is still part of the test system.

So the data is now sent from Node-RED either directly to the InfluxDB in the same LAN using the downloadable InfluxDB nodes or it is sent to Mosquitto through the built-in MQTT nodes. Mosquitto is the used MQTT broker which is installed in version 2.0.15 on a second VM with the exact same hardware specifications as the first VM running InfluxDB. Now there are the already described opportunities to either forward the data directly to the InfluxDB or sent it further to Kafka. To do so a connector is required in both cases. Here Telegraf as part of the InfluxData open-source software is used. It has the ability to forward data from a MQTT broker to

InfluxDB or Kafka and it is also possible to send data from Kafka to InfluxDB. Thus, Telegraf brings all functions needed in this test. It is used in the release 1.24.2. Kafka itself is running as version 3.3.1 on the second VM in parallel to Mosquitto. To get a better overview which VM is running which software and how these are connected Fig. 7 shows a schematic of this WAN part.

In order to now simulate the WAN the Raspberry Pi is separated from the VMs in the LAN. The connection is established from the Pi through a LTE hotspot and the public internet back to the LAN. This is representative since there are situations where it is not possible to use the network infrastructure of the production building itself. Therefore a connection over LTE/4G or even 5G is advisable.

Fig. 8 now shows the results of the performed test. On the x axis are the three mentioned data paths and on the y axis is a time scale in milliseconds. It becomes very clear that with an average of only 7 ms, the connection in the LAN has the lowest latency. The following is the WAN alternative via Mosquitto but without Kafka with an average time of 116 ms. The slowest way is via Mosquitto and Kafka in the WAN with an average of 312 ms.

From these data it can be concluded that Kafka has the highest impact on the latency. The gap between only Mosquitto and Mosquitto and Kafka is with 196 ms relatively high. This means that very careful consideration must be given to whether or not a Kafka implementation is necessary. In general, it is advisable to omit unneeded software and thus save yourself additional updates and error vulnerabilities. On the other hand, Kafka also offers advantages, as described in the previous chapter. Depending on the application, the structure of the system can be adapted to the requirements at this point.

V. CONCLUSION

The system presented in this paper, as shown, can be used for process monitoring in an industrial environment. It also enables entry into the production management of the future, which in many cases is based on automated data acquisition. Consequently, technologies such as enhanced OLAP presented in [17] can be used to process the data collected by the proposed system, for example. But even today, with the transformation of the fourth industrial revolution currently underway, this activity of digital surveillance has the highest relevance. The underlying potential for process optimization, scrap reduction and quality improvement, for example, is enormous. That's exactly why many companies are picking up on this movement and taking action themselves.

Immediately available ready-made systems from a wide range of suppliers on the market can be a solution here. However, as explained at the beginning, these also bring disadvantages, which must be carefully weighed. This paper provides the basis for an alternative approach. Important aspects such as data security, unwanted dependencies or lack of flexibility make this system stand out.

Thanks to the absence of external cloud providers, complete, standalone hosting of the software is possible. For

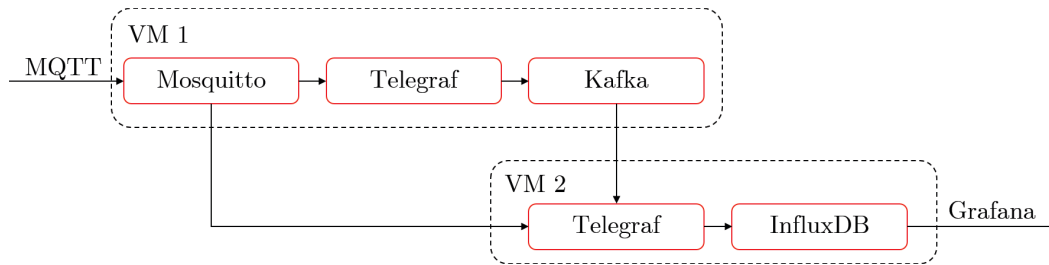


Fig. 7. Schematic WAN structure

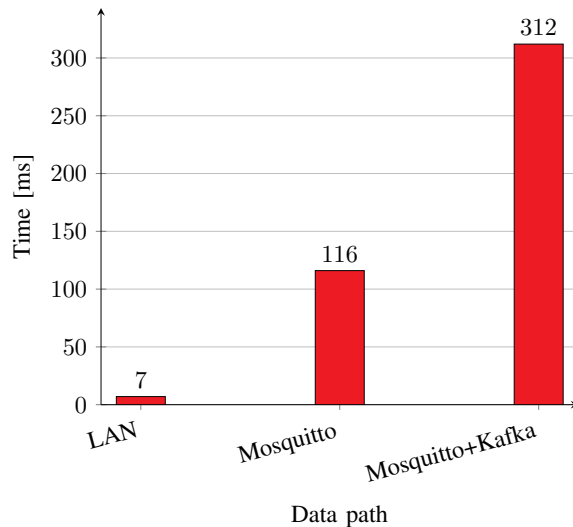


Fig. 8. Latency comparison of data paths

this purpose, individual components, such as the visualization software, can be flexibly exchanged, should it be necessary in the future. The possibility to make first test attempts in an environment with a free open-source license makes it easier to get started.

The benchmarks show why it is important to use the right type of database. Accordingly, it is worthwhile to use a special timeseries database for storing time-based data.

The latency of data transmission, which is not insignificant in this context, was also considered on the basis of some measurements. What stands out is the difference between a communication in the local network and a transmission over the public Internet. In addition, the delay caused by implementing a streaming service like Kafka should not be underestimated.

However, both the test of database types and that of latency offer potential for further investigation. For example, further comparisons can be made between other SQL databases and InfluxDB or other time series databases. The aim is to find out whether the results presented here can be confirmed or not.

The latency test must also be considered from other points of view. For example, what latency is necessary to enable

direct intervention in the process on the collected data basis? Is transmission on the WAN possible in this case or must this be done locally? This is only an excerpt from many other possible questions.

Overall, the goal is to answer these and to further optimize the system presented. This also includes the expansion to further machines and production sites worldwide. This upscaling presents the entire system with new challenges that will have to be overcome in the future.

REFERENCES

- [1] T. Devezas, J. Leitão, and A. Sarygulov, *Industry 4.0*, Cham, Switzerland: Springer, 2017, pp. 1–2.
- [2] F. Almada-Lobo, "The Industry 4.0 revolution and the future of Manufacturing Execution Systems (MES)," in *Journal of Innovation Management, Critical Manufacturing*, 2015, 18–19.
- [3] S. Winkler, *The new freedom in engineering: the ctrlX automation platform*, Bosch Rexroth, 2021, pp. 1–5.
- [4] J. Slemmons Stratford, J. Stratford, "Data Protection and Privacy in the United States and Europe," presented at the IASSIST Conference, New Haven, Connecticut, USA, May 21, 1998.
- [5] OpenJS Foundation & Contributors. "About:Node-RED." *nodered.org*. <https://nodered.org/about/#history> (accessed Sep. 20, 2022).
- [6] M. Lekić and G. Gardašević, "IoT sensor integration to Node-RED platform," presented at the 17th International Symposium INFOTEH-JAHORINA, East Sarajevo, Bosnia and Herzegovina, Mar. 23–23, 2018.
- [7] S. Chanthakit and C. Rattanapoka, "MQTT Based Air Quality Monitoring System using NodeMCU and Node-RED," presented at 7th ICT International Student Project Conference, Nakhonpathom, Thailand, Jul 11–13, 2018.
- [8] M. Tabaa, B. Chouri, S. Saadaoui and K. Alami, "Industrial Communication based on Modbus and Node-RED," presented at the 9th International Conference on Ambient Systems, Networks and Technologies, Porto, Portugal, May 8–11, 2018.
- [9] S. N. Z. Naqvi and S. Yfantidou, "Time Series Databases and InfluxDB," presented at Advanced Databases, Brussels, Dec. 17, 2017.
- [10] M. Nasar and M. A. Kausar, "Suitability Of Influxdb Database For Iot Applications," *International Journal of Innovative Technology and Exploring Engineering (IJITEE)*, vol. 8, pp. 1850–1857, Aug., 2019, doi: 10.35940/ijitee.I9225.0881019.
- [11] M. Chakraborty and A. P. Kundan, "Grafana," in *Monitoring Cloud-Native Applications*, New York, USA: Springer Science+Business Media, 2021, ch. 6, pp. 187–240.
- [12] S. Ren, J. S. Kim, W. S. Cho, S. Soeng, S. Kong, and K. H. Lee, "Big Data Platform for Intelligence Industrial IoT Sensor Monitoring System Based on Edge Computing and AI," presented at the 3rd International Conference on Artificial Intelligence in Information and Communication (ICAIC), Jeju Island, Korea, Apr. 13–16, 2021.
- [13] H. L. Manohar and T. R. G. Asir, "Data Consumption Pattern of MQTT Protocol for IoT Applications," in *Smart Secure Systems – IoT and Analytics Perspective*, G. P. Venkataramani, K. Sankaranarayanan, S. Mukherjee, K. Arputharaj, and S. S. Narayanan, Eds., Singapore, Singapore: Springer Nature, 2018, pp. 12–22.

- [14] A. Ledoul, G. S. Millan, A. Savulescu, and B. Styczen, "Data Streaming With Apache Kafka For Cern Supervision, Control And Data Acquisition System For Radiation And Environmental Protection," presented at 17th Int. Conf. on Acc. and Large Exp. Physics Control Systems, New York, NY, USA, Oct. 5-11, 2019.
- [15] InfluxData. "Optimize writes to InfluxDB." <https://docs.influxdata.com/influxdb/v2.4/write-data/best-practices/optimize-writes/#:~:text=InfluxDB%20scrapers-,Batch%20writes,5000%20lines%20of%20line%20protocol.> (accessed Oct. 18, 2022).
- [16] NTP.org. "5. How does it work?." <http://www.ntp.org/ntpfaq/NTP-s-algo.htm#Q-ACCURATE-CLOCK>
- [17] M. Usman, S. Asghar, and Simon Fong, Eds., "Integrated Performance and Visualization Enhancements of OLAP Using Growing Self Organizing Neural Networks," in *Journal of Advances in Information Technology*, Feb. 2010.